

Your passwords live on your phone.

A key in your pocket does the rest.

Safely is a password vault on your iPhone, a thumbnail-sized Bluetooth key (a Seeed Studio XIAO ESP32C3), and a Chrome extension. Open a login page with the key and phone nearby and the form fills itself. Walk away, and the browser knows nothing — because it never stored anything.



1 The phone

Holds every login in one AES-256 encrypted file. The key to it never leaves the iPhone. Light, animated SwiftUI app with Face ID, approvals and an activity log.

2 The key

A BLE relay the size of a stamp. Phone and computer both connect to it. It forwards sealed messages, stores nothing and can read nothing.

3 The browser

A Chrome extension spots login forms, asks the phone through the key, and fills. A tiny helper keeps the Bluetooth link alive and reconnects when you return.

This guide: 2 · How it works 3 · Why it is safe 4 · Set it up 5 · Test it, ship it, fix it

How it works

Four pieces pass one sealed message along. Only the two ends — the extension and the phone — hold the key that opens it.



What happens when you open a login page

- 1 The extension sees a password field.** It takes the site's address from Chrome itself — a page cannot lie about where it is.
- 2 It seals a request and hands it to the helper.** “Logins for <https://github.com>, please.” Encrypted before it leaves the browser.
- 3 The key relays it to your phone.** iOS wakes the app in the background over Bluetooth, even with the screen off.
- 4 The phone decides.** It matches the domain against the vault, applies your rule (fill automatically, or ask with Face ID), logs it, and seals the reply.
- 5 The form fills.** One match fills on its own; several show a small chooser. The password lives in page memory only.



Walk away and it stops

Bluetooth reaches a few metres. Take the key or the phone with you and the chain breaks; the extension holds no passwords to fall back on. Come back and both links reconnect by themselves — no tap needed.

Three things that make it feel seamless

It finds you

Phone and computer each keep a standing Bluetooth request for the key. The moment it is in range again, both links come back — nothing to tap, no app to open.

It works while asleep

iOS wakes Safely in the background when the key delivers a message, so the phone can stay locked in your pocket while the form fills.

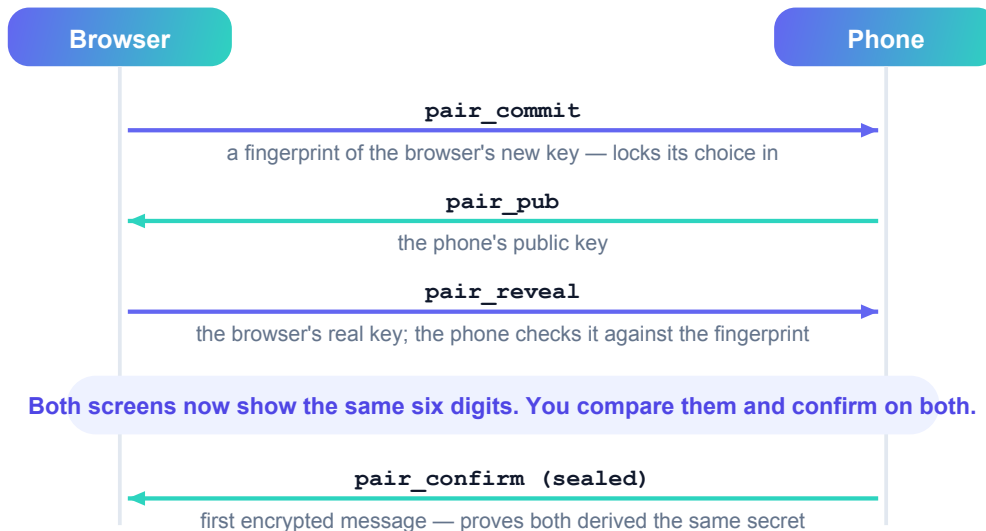
It learns new logins

Sign in somewhere by hand and the extension offers to save the login — to the phone, through the key. Chrome's own password manager is not involved.

Why it is safe

The design assumes the worst about everything in the middle. The key, the helper and the radio link are all treated as hostile; they only ever carry sealed envelopes.

Pairing: six digits that defeat a man in the middle



Because the browser commits to its key before it sees the phone's key, an attacker sitting between them cannot shop for a key that makes the digits collide. They get one blind guess in a million — and you would see different codes.

Who can see what

PART	WHAT IT HOLDS	WHAT IT NEVER HAS
● Chrome extension	A non-extractable session key. Passwords only for the page you are on, only in memory.	Nothing on disk. No vault.
● Bluetooth helper	Sealed envelopes passing through.	No keys. Cannot decrypt.
● Safely Key	Sealed frames for a few milliseconds.	No storage. Lose it: flash a new one for a few dollars.
● iPhone app	Everything — that is its job.	AES-256 file; key in the Keychain, this device only, never iCloud.

Also built in

- A rising counter in every message — recorded traffic cannot be replayed.
- Logins are matched by registrable domain; `evil.github.io` never sees `you.github.io`.
- 40 requests a minute per browser, every fill in the Activity log.
- Clipboard copies expire after 60 s and stay off Handoff.

Honest limits of version 1

- “Fill automatically” treats proximity as consent. Choose “Ask me every time” for Face ID on each request.
- No forward secrecy yet: a stolen browser profile plus recorded radio traffic could be decrypted.
- No cloud copy by design — use Settings → Export a backup.

Set it up

1 Power the key

Your XIAO ESP32C3 is already flashed with the Safely firmware and advertising as **Safely Key**. Plug it into any USB power — a battery pack in your bag works. To re-flash after a code change:

```
firmware/flash.sh
```

2 Install the helper and the extension

```
scripts/install-host.sh
```

Then in Chrome open `chrome://extensions`, switch on **Developer mode**, click **Load unpacked** and choose the `extension/` folder. The setup page opens by itself. If macOS asks whether Chrome may use Bluetooth, allow it.

3 Put the app on your iPhone

Open `ios/Safely.xcodeproj`, pick your iPhone, press Run — or install it from TestFlight (next page). Swipe through the welcome screens and allow Bluetooth. The Devices tab shows the key lighting up.

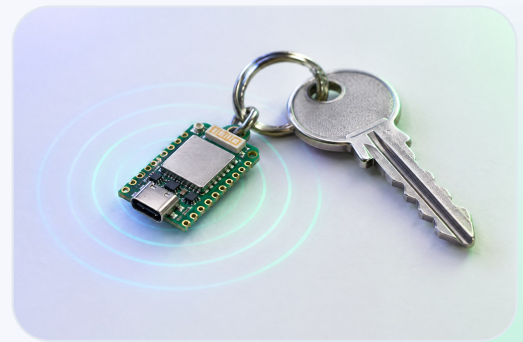
4 Pair

Phone: **Devices** → **Pair a browser**. Chrome: **Start pairing**. Both show six digits. If they are the same, confirm on both. That is the only time you compare anything.

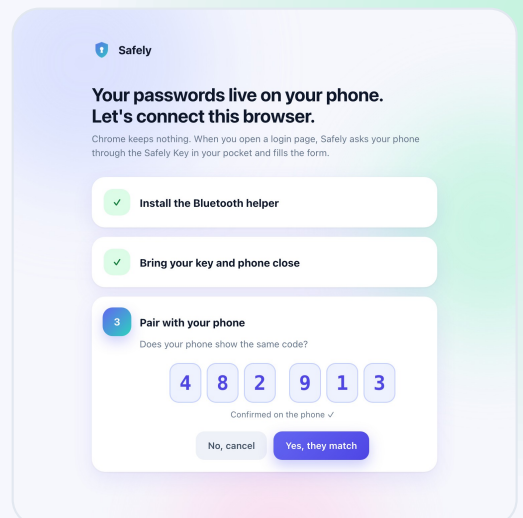
5 Bring your passwords over

Chrome: `chrome://password-manager/settings` → **Export passwords**. Safari: `File` → `Export` → `Passwords`. Click the Safely icon → **Import passwords** and drop the file in. It is encrypted in the browser and travels through the key to the phone — 250 logins take well under a minute.

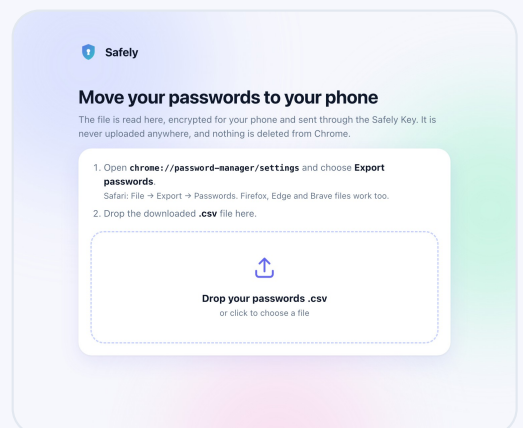
Nothing is deleted from Chrome. Delete the `.csv` afterwards (it is plain text), and clear Chrome's saved passwords yourself once you trust Safely.



The key: a XIAO ESP32C3 on USB power.

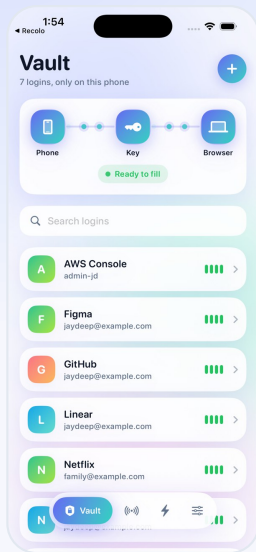


The extension's setup page during pairing.

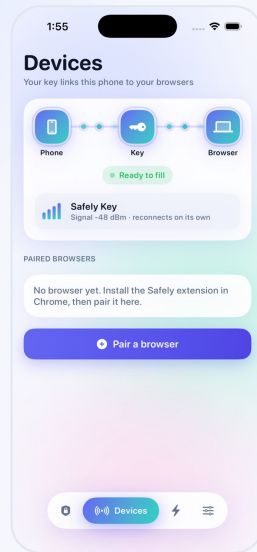


Import: drop the exported `.csv`.

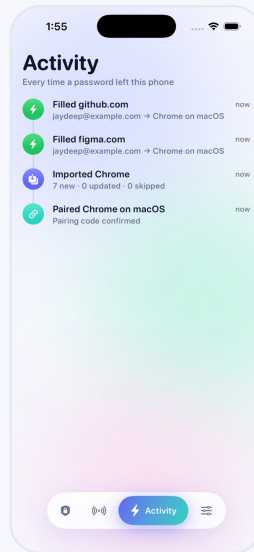
Test it, ship it, fix it



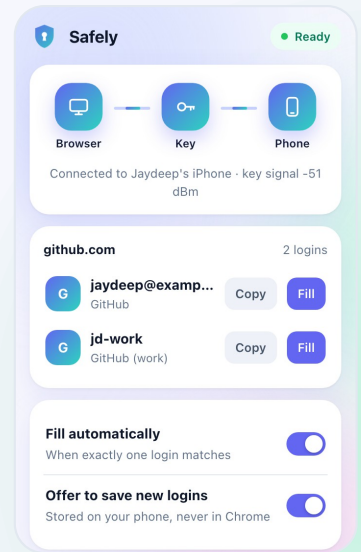
Vault



Devices



Activity



Chrome popup

Try it

- 1 First fill.** Run `scripts/serve-test-page.sh`, add a login for `http://localhost:8765` in the app, open the page. The fields glow and fill.
- 2 Walk away.** Carry the key out of the room. The popup turns to "Looking for key"; a reload fills nothing. Come back: "Ready" within seconds.
- 3 Ask every time.** Settings → "Ask me every time". Reload: the phone asks, Face ID approves. Try Deny as well.
- 4 Save a new login.** Type a login by hand on a site Safely does not know. A "Save this login to Safely?" card appears.
- 5 Fill inside iPhone apps.** iOS Settings → General → AutoFill & Passwords → turn on Safely. Any app's login now offers your vault.

No iPhone at hand?

A simulator runs the phone's exact protocol code on the Mac with four demo logins:

```
cd core && swift build
.build/debug/safely-simphone
```

Automated checks

```
cd core && swift test
node scripts/protocol-test.mjs
node scripts/ble-e2e-test.mjs
```

Unit tests · extension JavaScript against phone Swift · the whole chain over real Bluetooth.

TestFlight

Once: sign in under Xcode → Settings → Accounts, and create the app in App Store Connect with bundle ID `com.codecrackjd.safely`. Then:

```
scripts/testflight.sh --upload
```

Without `--upload` it writes `build/export/Safely.ipa` for the Transporter app.

If something is off

"Helper missing" — Run `scripts/install-host.sh`, then reload the extension.

"Phone away" — Open Safely once on the phone; iOS then keeps the link in the background.

"Looking for key" — Is the key powered? System Settings → Privacy → Bluetooth → Chrome on.

Nothing fills — Does the site's address in the vault match? Check the phone's Activity tab.

Helper log: `~/Library/Logs/Safely/host.log` · Key log: `arduino-cli monitor -p /dev/cu.usbmodem1101 -c baudrate=115200` · Protocol: `docs/PROTOCOL.md`